



VERIFIZIERTE C-LIBRARY FÜR LOCKFREIE DATENSTRUKTUREN

ENTWICKLUNG DER LOCKFREIEN DATENSTRUKTUREN

Software-Entwicklungspraktikum (SEP)

Sommersemester 2023

Angebot

Auftraggeber

Technische Universität Braunschweig

Institute of Theoretical Computer Science

Prof. Dr. rer. nat. Roland Meyer

IZ 346, i.e. Informatikzentrum, 3rd floor, office 346, Mühlenpfordtstr. 23

D-38106 Braunschweig

Betreuer: Sören van der Wall

Auftragnehmer:

Name	E-Mail-Adresse
Keanu Wirz	k.wirz@tu-braunschweig.de
Nazli Cesur	n.cesur@tu-braunschweig.de
Tristen Vaeckenstedt	t.vaeckenstedt@tu-braunschweig.de
Jacob Müller	jacob.mueller@tu-braunschweig.de
Mohamed Amine Halila	m.halila@tu-braunschweig.de
Omar Zitouni	o.zitouni@tu-braunschweig.de

Braunschweig, 25. April 2023

Bearbeiterübersicht

Kapitel	Autoren
1	Keanu, Tristen
1.1	Keanu, Tristen
1.2	Keanu, Tristen
2	Jacob
3	Omar, Mohamed Amine
3.1	Omar, Mohamed Amine
3.2	Omar, Mohamed Amine, Keanu
3.3	Keanu
4	Nazli, Keanu
4.1	Nazli, Keanu
4.2	Nazli, Keanu
4.3	Nazli, Keanu
5	Mohamed Amine, Omar
5.1	Mohamed Amine, Omar
5.2	Mohamed Amine, Omar
5.3	Mohamed Amine, Omar
6	Nazli, Keanu
6.1	Nazli, Keanu
6.2	Nazli, Keanu
6.3	Nazli, Keanu
7	Keanu, Tristen, Jacob

Inhaltsverzeichnis

1	Einleitung	5
1.1	Ziel	5
1.2	Motivation	5
2	Formale Grundlagen	6
3	Projektablauf	7
3.1	Dokumentabgaben	7
3.2	Coding Meilensteine	8
3.3	Geplanter Ablauf	9
4	Projektumfang	10
4.1	Lieferumfang	10
4.2	Kostenplan	10
4.3	Funktionaler Umfang	10
5	Entwicklungsrichtlinien	11
5.1	Konfigurationsmanagement	11
5.2	Design- und Programmierrichtlinien	11
5.3	Verwendete Software	11
6	Projektorganisation	12
6.1	Schnittstelle zum Auftraggeber	12
6.2	Schnittstelle zu anderen Projekten	12
6.3	Interne Kommunikation	12
7	Glossar	13

Abbildungsverzeichnis

3.1	Gantt-Diagramm für unseren Ablauf	9
-----	---	---

1 Einleitung

1.1 Ziel

Das Ziel dieses Projekts ist es, eine verifizierte C-Library für lockfreie Datenstrukturen zu entwickeln, die nicht auf Threadlocking für Synchronisation setzt. Stattdessen soll eine optimiertere leistungsschnellere aber dennoch threadsichere Alternative entwickelt werden, die bugfrei und leistungsoptimiert ist.

Am Ende des Projektes wird es eine Library mit den Datenstrukturen List, Queue, Set und Stack geben, die mit schwacher Synchronisation benutzbar sind. Unter anderem wird es einige visualisierte Benchmark-Tests geben, um den Unterschied zwischen der lockfreien Library und einer Library mit Semaphoren zu verdeutlichen.

1.2 Motivation

Synchronisation von Threads ist von der heutigen Zeit nicht mehr wegzudenken. Da Fehler bei Synchronisation sehr schwerwiegend sind, wird viel Wert darauf gelegt diese zu vermeiden. Eine schon geprüfte und verifizierte Library von Datenstrukturen erleichtert einem dieses Vorhaben enorm. Denn der Programmierer muss sich nicht selbst um die Synchronisation kümmern und braucht die Zugriffe nicht zu überprüfen. Insbesondere in leistungsschwachen System bietet es sich an auf lockfreie Datenstrukturen zurückzugreifen um die Performance zu verbessern.

2 Formale Grundlagen

In diesem Kapitel sollen die formalen Grundlagen der zu entwickelnden Software aufgeführt werden.

Als Programmiersprache wird in diesem Projekt C verwendet, um genau zu sein Version C11. Bezüglich der Projektumgebung haben wir uns dazu entschlossen, dieses Softwareprojekt systemübergreifend laufen zu lassen. Die C-Library soll unter Linux, Mac OS und Windows funktionieren, somit auch auf den Prozessorarchitekturen AMD64 und ARM, ohne Performance zu verlieren.

Zum kompilieren wird der Compiler GCC verwendet, desweiteren wird aufgrund von essenzieller Wichtigkeit für die Software, von der C-Atoms Library Gebrauch gemacht. Dies hat den Grund, dass ein atomarer Datentyp von mehreren Threads gleichzeitig gelesen/verändert werden kann, ohne dass ein data-race entsteht.

Die zu entwickelnden Algorithmen für die lockfreien Datenstrukturen werden nicht komplett neu entwickelt, sondern basieren auf vorhandenen Methoden. Für die Visualisierung nutzen wir das Java Framework JavaFX, diese erstellt uns die Grafiken die zur Präsentation und in unsrem Visualisierungsprogramm genutzt werden.

3 Projektablauf

Die Abgabetermine und die zugehörige Bezeichnung des Meilensteins werden in der folgenden Tabelle und in der Abbildung 3.1 dargestellt. Vor dem Abgabetermin müssen die Dateien rechtzeitig in das GitLab-Repository hochgeladen werden.

3.1 Dokumentabgaben

Nummer	Meilenstein	Dokumente	Abgabetermin
1	Projektstart	-	17.04.23
2	Angebot	Angebot	26.04.23
3	Pflichtenheft	Pflichtenheft	17.05.23
4	Abnahmespezifikation	Abnahmespezifikation	17.05.23
5	Zwischenpresentation	-	26.05.23
6	Fachentwurf	Fachentwurf	07.06.23
7	Technischer Entwurf	Technischer Entwurf	28.06.23
8	Testdokumentation	Testdokumentation	12.07.23
9	Tag der jungen Software EntwicklerInnen	-	20.07.23

3.2 Coding Meilensteine

Nummer	Meilenstein	Starttermin	Endtermin
1	Organisation und Überblick	17.04.23	26.04.23
2	Verständnis und erstimplementation Algorithmen	27.04.23	04.05.23
3	Grobentwurf: erster Prototyp	05.05.23	26.05.23
4	Feinentwurf	27.05.23	10.06.23
5	Implementierung der Tests	11.06.23	28.06.23
6	Vorbereitung der Präsentation	29.06.23	19.07.23
7	Präsentation	20.07.23	20.07.23

Abbildung 3.1: Gantt-Diagramm für unseren Ablauf

4 Projektumfang

Dieses Kapitel dient dazu, den Projektumfang zu definieren.

4.1 Lieferumfang

Die Library enthält die Datenstrukturen Stack, Queue, List und Set. Diese sollen außerdem mit Benchmark und Tests geliefert werden. Unter anderem stellen wir eine graphische Darstellung bereit die den Unterschied zwischen Lock free und mit Mutex und Co darstellt. Geliefert wird am Ende des Projektes der komplette Quellcode für die Library, Tests und Benchmark, eine ausführbare Anwendung zur Visualisierung der Performance sowie die Dokumente Angebot, Pflichtenheft, Abnahmespezifikation, Fachentwurf, Technischer Entwurf und Testdokumentation.

4.2 Kostenplan

Jedes Projektmitglied soll insgesamt circa 210 Stunden arbeiten, um das Projekt erfolgreich abzuschließen. Wir beziehen uns in der Berechnung auf ein Stundenlohn von 100 Euro. Jedes Mitglied kriegt dementsprechend $210 \cdot 100 \text{ Euro} = 21.000 \text{ Euro}$ für das gesamte Projekt. Unser Team besteht aus 6 Softwareentwickler:innen. In der Summe kostet das ganze Projekt $21.000 \text{ Euro} \cdot 6 = 126.000 \text{ Euro}$.

4.3 Funktionaler Umfang

Die Datenstrukturen sollen Zugriffsfunktionen wie Einfügen oder Entfernen von Daten bereitstellen, außerdem sollen diese Zugriffe ohne Verwendung von Locks threadsicher sein. Die Tests und der Benchmark soll garantieren, dass das Program bugfrei und optimiert läuft. Im Zuge der Benchmarks wird es ein Programm geben, welches eine Grafik erstellt die den Vorteil von Lock free data structures darstellen soll. Dieses Programm soll auch Variablen besitzen um beispielsweise die Anzahl der Zugriffe zu erhöhen. Letztendlich soll die Library korrekt mit den C Standard Algorithmen (sorting Algorithmen zum Beispiel) funktionieren.

5 Entwicklungsrichtlinien

5.1 Konfigurationsmanagement

Es wurde uns schon ein Git-Repository zur Verteilung der Codedateien bereitgestellt. Dabei haben wir eine Datei pro lockfreie Datenstrukturen, die Include-, Source- und Test-Dateien enthalten. Jeder Commit, der auf den Master-Branch gepusht wird, soll ausführbar sein und mit einem beschreibenden Kommentar bezüglich der veränderten oder neuen Funktionen ausgestattet sein.

5.2 Design- und Programmierrichtlinien

Damit ein Projekt wie dieses stets übersichtlich bleibt, sollte darauf geachtet werden, dass alle Formatierungsrichtlinien eingehalten werden und man ausführlich dokumentiert, dafür halten wir uns an die Regeln von CLion Format und auf Englisch geschriebene Doxygen Kommentare.

5.3 Verwendete Software

Für unser auf C Programmiersprache geschriebenes Projekt wird CLion benutzt, die eine integrierte Entwicklungsumgebung ist und von Jet Brains entwickelt wurde. Außerdem werden Unit Tests, die eine Art automatisierter Softwaretests ist, benutzt um einzelne Einheiten oder Komponenten einer Softwareanwendung zu überprüfen. Zusätzlich werden Benchmarks verwendet, die eine Methode zur Bewertung der Leistung von Computerhardware und -software. Dabei werden spezielle Testprogramme eingesetzt, die die Leistung des Systems in Bezug auf die Zeit, die zur Ausführung der Tests benötigt wird, messen. Für das Gantt-Diagramm wird Excel, das ein Tabellenkalkulationsprogramm von Microsoft, mit dem Benutzer Daten in Tabellenform organisieren können, ist, benutzt. Um die Diagramme zu erstellen, werden wir UMLet verwenden, der ein kostenloses, Open-Source UML (Unified Modeling Language) ist.

6 Projektorganisation

In diesem Kapitel soll die Kommunikation des Teams festgehalten werden.

6.1 Schnittstelle zum Auftraggeber

Unseren Betreuer Söhren van der Wall erreichen wir über Email und über die wöchentlichen Meetings. Außerdem ist uns bekannt wie wir ihn in Person in seinem Büro erreichen können.

6.2 Schnittstelle zu anderen Projekten

Das Team TCS3, welches für die Optimierung mittels Dartagnan verantwortlich ist wird unsere Implementierung auf Bugs prüfen und untersuchen.

6.3 Interne Kommunikation

Das Team trifft sich 2 mal die Woche sowohl Online als auch in Person. Zur internen Kommunikation wird Whatsapp, Discord und E-Mail genutzt. Die Projektorganisation übernimmt Keanu Wirz, dies beinhaltet zum Beispiel Aufgabenverteilung und Organisation der Meetings. Probleme werden in Absprache mit den anderen gelöst, gegebenenfalls mit Hilfe und Unterstützung des ganzen Teams.

7 Glossar

Hier werden Fachbegriffe erklärt.

Begriff	Definition
AMD64	Implementierung einer Prozessorarchitektur
ARM	Implementierung einer Prozessorarchitektur
Benchmark	Vergleichende Analyse von Ergebnissen oder Prozessen
Betriebssystem	Zusammenarbeit von Software und Hardware zur Abarbeitung von Programmen
Bugfrei	Fehlerfrei
C-Atomic Library	Bibliothek, die es erlaubt atomare Datentypen zu erzeugen
C-Library	Standardbibliothek der Programmiersprache C
Compiler	Computerprogramm, welches Quellcode in eine für den Computer lesbare Form übersetzt
Dartagnan	Tool zur Erkennung und Behebung von Bugs sowie Optimierungen
Data-race	Threads greifen zur selben Zeit auf eine gemeinsame Variable zu
Datenstruktur	Objekt, in dem Daten gespeichert werden
Discord	Instant Messenger
GCC	Compiler von GNU-Projekt
Git	Ein Versionskontrollsystem zur Verwaltung von Änderungen an einem Code, um die Zusammenarbeit von Entwicklern zu erleichtern
Library	Sammlung ähnlicher Objekte
Linux	ein freies Open Source-Betriebssystem
Lock free	Ohne Verwendung von Locking Algorithmen
Locking Algorithmus	Algorithmus der zur Synchronisation von Threads verwendet wird und diese sperrt/freigibt (lock/unlock)
Mac OS	Von Apple entwickeltes Betriebssystem
Natürliche Sprachen	Eine von Menschen gesprochen Sprachen
Prozessorarchitektur	Aufbau von Prozessoren
Thread	Aktivitätsträger in der Abarbeitung eines Programms (Teil eines Prozesses)
TCS	Institut Theoretical Computer Science
Whats App	Instant Messenger
Windows	Von Microsoft entwickeltes Betriebssystem